Constant-time Quaternion Algorithms for SQIsign

Andrea Basso[1], Chenfeng He[2], David Jacquemin[3,4], Fatna Kouider[2], Péter Kutas[2,5], Anisha Mukherjee[3], Sina Schaeffler[1,6], and Sujoy Sinha Roy[3]

[1] IBM Research Europe, Zürich, Switzerland;
[2] Eötvös Loránd University, Budapest, Hungary;
[3] Graz University of Technology, Graz, Austria;
[4] Institute for Infocomm Research, A*STAR, Singapore;
[5] University of Birmingham, Birmingham, United Kingdom;
[6] ETH Zurich, Zurich, Switzerland
andrea.basso@ibm.com, {chenfenghe,fatnakouider}@inf.elte.hu, david.jacquemin@student.tugraz.at, kutasp@gmail.com, {anisha.mukherjee,sujoy.sinharoy}@tugraz.at, sschaeffle@ethz.ch

Abstract. SQIsign, the only isogeny-based signature competing in the ongoing NIST call for additional signatures, offers the most compact key and signature sizes among all other candidates. It combines isogenies with quaternion arithmetic for its signing procedure. In this work, we address a gap in the current implementation of SQIsign: the absence of constant-time algorithms for quaternion arithmetic. We propose constant-time algorithmic formulations for three fundamental routines in SQIsign's quaternion layer.

First, we discuss a constant-time Hermite Normal Form (HNF) algorithm. We then present a new constant-time approach for computing a generator of a quaternion ideal, replacing the exhaustive search-based approach used in SQIsign. Our approach eliminates the need for coefficient scanning, coprimality tests, and norm evaluation loops, yielding a data-independent and deterministic procedure. Finally, we design a constant-time version of the GeneralizedRepresentInteger algorithm for solving norm equations in special extremal orders. We circumvent timing dependencies arising from primality checks, modular square root calculations, and Euclidean division steps by introducing a regularized control flow with fixed-iteration sampling and branch-free arithmetic. We also show that the tools developed along the way enable a constant-time version of the recently introduced Qlapoti algorithm.

In our constant-time algorithms, the cost of large operand operations remains a bottleneck for the constant-time HNF and GeneralizedRepresentInteger. We believe our work will facilitate secure and efficient implementations and inspire further works on deployment-level optimizations.

Keywords: Isogeny-based cryptography · SQIsign · Side-channel analysis · Isogeny-based signature · Constant-time implementation

Author list in alphabetical order; see https://ams.org/profession/leaders/CultureStatement04.pdf.

1 Introduction

Post-quantum cryptography (PQC) has taken center stage in mitigating the quantum threat posed by algorithms such as Shor's [29, 30], which efficiently solve the integer factorization and discrete logarithm problems that underlie the security of classical schemes such as RSA and elliptic curve cryptography. Among the various paradigms explored in PQC, isogeny-based cryptography has attracted considerable interest. Isogeny-based schemes such as SIDH [14, 17], SIKE [16] were notable submissions to the NIST PQC standardization process. More recently, in response to NIST's call for additional digital signature schemes, the only isogeny-based candidate, SQIsign [2], has attracted substantial attention for achieving the smallest known public key and signature sizes across all post-quantum signature proposals.

SQIsign relies on the presumed hardness of computing endomorphism rings of supersingular elliptic curves. The scheme is constructed using the Deuring correspondence, which establishes a correspondence between supersingular elliptic curves over $\mathbb{F}_{p^2}$ and maximal orders of a quaternion algebra $\mathcal{B}_{p,\infty}$ ramified at p and infinity. In SQIsign, the public key is a supersingular elliptic curve, and the signature serves as a non-interactive proof of knowledge of the associated left ideal in a quaternion order. Realizing this correspondence in practice requires efficient algorithms to translate between quaternion and elliptic curve domains. This translation is computationally expensive and hinges on constructing ideals (or isogenies) with norms (or degrees) that are smooth enough to allow efficient computation. Early one-dimensional variants of SQIsign, while compact and elegant, suffered from high signing latency due to the cost of this ideal-to-isogeny translation. Moreover, scaling the scheme to higher security levels introduced additional difficulties as the requirements on the choice of prime became more restrictive with growing security parameters. These limitations were a major bottleneck in the practical deployment of one-dimensional SQIsign variants.

Following the attacks [7, 24, 26] on SIDH and SIKE, the underlying attack strategy was adapted for constructive use in protocols involving higher-dimensional isogenies. These developments led to a family of SQIsign variants that embed elliptic curve isogenies into isogenies between higher-dimensional abelian varieties, thereby improving the efficiency of signature generation. Notably, SQIsignHD [12] introduced the use of eight-dimensional isogenies to overcome the aforementioned drawbacks and achieve improved theoretical security. More recent proposals, including SQIsign2D-East [25] and SQIsign2D-West [4], restrict the construction to two-dimensional isogenies, striking a balance between security, efficiency and compactness. In line with these advances, the updated NIST Round 2 specification of SQIsign [2] incorporates the algorithmic structure of the two-dimensional variants.

Motivation and related work. The official SQIsign specification explicitly states that its current implementation is not constant-time, with the exception of selected components on the isogeny side, such as finite field, elliptic curve, and pairing arithmetic. In SQIsign, the majority of secret-dependent computa-

tions are performed within the quaternion algebra, often as operations on or among ideals, before being translated into their corresponding isogenies. For instance, during key generation, the secret signing key is represented by a randomly sampled left ideal in the maximal order of the quaternion algebra, and the corresponding secret curve is obtained only through the translation of this ideal into an isogeny. Similarly, in the commitment phase, the ephemeral randomness is generated as an ideal and subsequently mapped to an isogeny. In the same way, the response steps are dominated by algebraic operations on the secret, commitment and challenge ideals, with an ideal-to-isogeny translation before the final signature generation. This observation highlights the fact that secret or secret-dependent data 'live' longer on the quaternion side in SQIsign than on the elliptic curve side. Consequently, a wide range of quaternion algorithms are executed on sensitive inputs. Following the NIST Round 2 SQIsign specification [2, Section 3.1], we list the core quaternion-based operations: big-integer arithmetic, basic integer linear algebra including computing Hermite Normal Form (HNF), operations on lattices including 4-dimensional lattice reduction, operations on ideals (multiplication, intersection, conjugation) and finding generators of ideals and, finding endomorphisms of a given curve which reduce to solving norm equations in special extremal maximal orders in the quaternion algebra.

Existing works only cover limited aspects of SQIsign's quaternion side and is essentially restricted to constant-time alternatives for integer arithmetic and lattice reduction. The recent work by [23] proposes and implements constant-time alternatives for fundamental big-integer functions such as integer addition, multiplication, Euclidean division, GCD and square-root computation. Another recent work by [18] establishes theoretical and experimental bounds on the size of these integers and proposes fixed-precision big-integer arithmetic in the context of SQIsign. Regarding quaternion algorithms using these big-integer operations, only 2- and 4-dimensional constant-time lattice reduction algorithms have been proposed by [15]. Thus, the rest of SQIsign's quaternion algorithms such as algorithms for finding generators of ideals and algorithms for norm representation still lack constant-time formulations.

1.1 Our contributions

In this work, we take a step toward closing this gap by designing constant-time algorithms and exploring implementation techniques for three key routines in SQIsign's quaternion layer: finding generators of ideals using the subroutine IdealGenerator [2, Algorithm 3.8], solving norm equations via the GeneralizedRepresentInteger [2, Algorithm 3.12] procedure, and computing Hermite Normal Forms via HNF [2, Algorithm 3.2]. Our contributions are as follows.

We propose a new constant-time algorithm to find a generator of a quaternion ideal I, thereby replacing the original non-constant time search-based approach [2, Algorithm 3.8] from the SQIsign specification. The prior approach incrementally searched for suitable ideal generators by iterating over increasingly large coefficient ranges along with performing coprimality checks (with nested GCD computations) and norm computations. This results in both high runtime

and data-dependent behavior. Instead, our proposed approach guarantees that, given an HNF basis of a primitive left ideal I (i.e. the standard representation of ideals in SQIsign), a specific element of the basis yields a generator of I together with the ideal norm (Theorem 1). This result allows us to bypass the exhaustive search entirely and compute a valid generator deterministically and in constant-time.

Next, we design a constant-time version of the GeneralizedRepresentInteger algorithm, a core subroutine in SQIsign used for solving norm equations in special extremal orders. The reference algorithm (which we outline in Algorithm 1) in the SQIsign specification [2] involves data-dependent control flow due to random sampling, primality testing, modular square root computations, and the use of Cornacchia's algorithm, all of which introduce significant timing variability. Our contribution is a complete reworking of this routine to operate in constant-time. We remove the need for primality testing and avoid conditional branching in the modular square root algorithm, which enable a deterministic algorithmic flow. We reformulate Cornacchia's algorithm to eliminate variable loop bounds and conditional branches by using a fixed number of steps and replacing variable-time Euclidean division with equivalent arithmetic operations that can be executed within the set iteration bound. Moreover, we also design a constant-time algorithm for computing the square root of an integer that is a square more efficiently than previous approaches [22] (both faster and requiring no fixed-point arithmetic). Finally, we also show that our developed methods can be used to produce a constant-time version of the norm equation solving step in the recently introduced Qlapoti algorithm [6].

As a supporting contribution, we also implement a constant-time algorithm for computing the HNF of full-rank integer matrices, tailored to SQIsign. While HNF computation poses no conceptual difficulty for constant-timeness assuming availability of a constant-time GCD routine, the challenge lies in handling very large integers that arise during the computation. We refer to [23] for constant-time big-integer arithmetic but do not use it in our implementation. Specifically, [23] slightly adapts the constant-time GCD algorithm introduced in [5] to work with large integers in SQIsign.

We note that this work focuses on formulating data-independent control flow and algorithmic frameworks for the aforementioned quaternion subroutines in SQIsign. We do not aim to provide highly optimized or "fast" implementations. Once such constant-time formulations are established, we believe that many implementation-level optimizations and tricks can be readily applied. We hope that this work will draw the attention of practitioners and implementation-focused cryptographers toward further refining and optimizing these routines for deployment.

1.2 Organization of the paper

We present the necessary mathematical background on quaternion arithmetic and the SQIsign protocol in Section 2. Our main contributions and observations

are detailed in the following three sections: Section 3 describes our constant-time approach for computing Hermite Normal Forms, Section 4 addresses the problem of finding generators of ideals in constant-time, and Section 5 outlines our constant-time solution for solving norm equations. Section 6 studies the novel ideal-to-isogeny algorithm Qlapoti from [6] with respect to constant-time behavior and describes a naive constant-time version of it. Remarks on our proof-of-concept implementation and its performance are in Section 7.

2 Preliminaries

In this section, we introduce the basic mathematical concepts needed to understand quaternion arithmetic and how they are used in the SQIsign protocol. We also give an overview of SQIsign, focusing on the key quaternion algorithms that we improve in this work. We denote the bitwise `AND`, right and left shift operations by the symbols '&', '$\ll$' and '$\gg$' respectively. The function ctCmov refers to the operation that performs a conditional selection and copy in constant-time, like shown in [23]. We define ctIsZero as a constant-time function that outputs `1` if the input is `0`, else, if the input is any other non-zero integer, the function outputs `0`.

2.1 Constant-time cryptography

A cryptographic implementation is said to be 'constant time' when its observable running time is *independent* of any secret data it processes. A common approach to achieving a constant-time implementation is to ensure that the algorithm follows an identical control-flow path for every possible input, so that the same sequence of operations is always executed. Another approach is to randomize or blind the sensitive data so that the timing of the resulting computation is statistically independent of the original secrets.

The constant-time literature generally assumes that the latency of the basic arithmetic and logical instructions of the processor, such as addition, subtraction, multiplication, bitwise `AND` and `XOR`, and shifts, is itself data-independent on the target microarchitecture. We refer to this as the 'constant-time assumption' for the targeted computing platforms. We also assume some simple side-channel free functions, such as constant-time conditional swaps `ctCondMov` and constant-time large-integer arithmetic available, including basic arithmetic ($+, -, =, \times$, euclidean division, extended greatest common divisor, ...) and a test of equality to zero denoted `ctIsZero`. This paper aims to make the quaternion part of the signing algorithm of SQIsign constant-time. It is important to note that the constant-time property does not provide complete side-channel immunity. Microarchitectural threats that observe cache-line activity or exploit speculative execution [20] might still extract information. More powerful physical side-channel attacks [21] exploiting power or electromagnetic leakage can still work against constant-time implementations. Such attacks are outside the scope of timing countermeasures.

2.2 Deuring correspondence

A quaternion algebra is a central simple algebra of dimension 4 over a field K. Whenever the characteristic of K is not 2, it has a presentation with a basis $1, i, j, ij$ with the relations $i^2 = a, j^2 = b,\ ij = -ji$ where $a, b \in K$ (where we identify the center of the algebra with K). A quaternion algebra is either a division algebra or is isomorphic to $M_2(K)$. When $K = \mathbb{Q}$, then isomorphism classes of quaternion algebras can be described by their completions. Namely, there is always a finite set of places where the completion is a division algebra (this number is always even by Hilbert reciprocity) and these places are called ramified places. The quaternion algebra relevant to isogeny-based cryptography is ramified at a prime p and at infinity. In most contexts one assumes that $p \equiv 3 \pmod 4$ and then it admits the presentation $i^2 = -1, j^2 = -p,\ ij = -ji$ and from now on we will always make this assumption and denote this quaternion algebra by $B_{p,\infty}$. For every element $\alpha = a + bi + cj + dij$, one can associate its conjugate $\overline{\alpha} = a - bi - cj - dij$. The norm of α is defined as $n(\alpha) = \alpha\overline{\alpha}$ and the trace of α is defined as $Tr(\alpha) = \alpha + \overline{\alpha}$. It can be easily seen that both the norm and trace are in $\mathbb{Q}$.

A maximal order in $B_{p,\infty}$ is a subring that contains 1 and is a full $\mathbb{Z}$-lattice. Deuring's theorem [13] states that endomorphism rings of supersingular elliptic curves are maximal orders in $B_{p,\infty}$; furthermore, every maximal order is isomorphic to the endomorphism ring of some supersingular elliptic curve (there are either two curves or one curve corresponding to a maximal order). Due to the fact that $B_{p,\infty}$ is a positive definite quaternion algebra, the norm map induces a lattice structure on maximal orders. Every maximal order is thus a 4-dimensional Euclidean lattice whose determinant is p^2.

Isogenies between elliptic curves will correspond to one-sided ideals. An isogeny $E_1 \to E_2$ corresponds to a left ideal of $\mathrm{End}(E_1)$ and a right ideal of $\mathrm{End}(E_2)$. The norm of an ideal is the greatest common divisor of all elements in the ideal: by the correspondence between ideals and isogenies, the norm of a left ideal is the degree of the corresponding isogeny. Every left ideal can be generated by two elements, one of which is the norm of the ideal.

2.3 High-level description of SQIsign

SQIsign is a signature scheme whose security relies on the hardness of computing the endomorphism ring of a supersingular elliptic curve. At its core lies a Σ protocol, which is then converted into a signature via the standard Fiat–Shamir transform.

We thus describe the Σ protocol: the protocol always works in the setting $p \equiv 3 \pmod 4$, thus the curve $E_0 : y^2 = x^3 + x$ is always supersingular and acts as a starting point. This is particularly convenient since its endomorphism ring is generated by $1, \iota, \frac{\iota+\pi}{2}, \frac{1+\iota\pi}{2}$, where ι is the endomorphism $\iota : (x, y) \to (-x, \sqrt{-1}y)$ and π is the $\mathbb{F}_p$-Frobenius. The corresponding maximal order is denoted by $\mathcal{O}_0$, and the quaternions associated with ι and π are i and j since $i^2 = -1$ and $j^2 = -p$.

Key generation consists of sampling a uniformly random elliptic curve. To do so, the protocol samples a random left $\mathcal{O}_0$-ideal I of sufficiently large norm, and then translates into the corresponding isogeny $\phi_I : E_0 \to E_{\mathsf{pk}}$. The ideal acts as the secret key of the protocol, which is represented by a generator, which needs to be computed on the fly. The ideal-to-isogeny translation is one of the main building blocks at the core of SQIsign and involves several subroutines, but one of its key elements is the sampling of endomorphisms of arbitrary degree, which corresponds to solving a norm equation. When the endomorphism belongs to E_0 or any supersingular curve whose endomorphism ring is a special extremal maximal order, this algorithm is known in the SQIsign literature as RepresentInteger. To underline that special extremal orders distinct from $\mathrm{End}(E_0)$ are also supported, the NIST Round 2 submission of SQIsign calls its implementation of this algorithm GeneralizedRepresentInteger. We follow this naming convention.

Commitment generation is similar to key generation, where the signer samples a random commitment curve E_{com}. It then receives a challenge in the form of an isogeny $\phi_{\mathsf{chl}} : E_{\mathsf{pk}} \to E_{\mathsf{chl}}$, and it is supposed to respond with an isogeny $\phi_{\mathsf{rsp}} : E_{\mathsf{chl}} \to E_{\mathsf{com}}$. To construct such a response, the signer computes an ideal whose left order is the order associated to E_{chl} and whose right order is the order associated to E_{com}: from this, it can sample a uniformly random isogeny between E_{chl} and E_{com}. Note that, unlike in the previous cases where the algorithms only worked with ideal classes, in this case the signer needs to give a representation of the response isogeny; thus, it needs to extract how the response isogeny acts on a torsion basis. To do this, the signer needs to compute a generator of the response ideal. Lastly, for efficiency reasons, the response isogeny of degree u is represented by a two-dimensional representation, which means that the signer needs to sample an auxiliary isogeny from E_{chl} of degree $2^a - u$, for some large a. This, once again, involves finding an endomorphism on E_0 of arbitrary norm, from which it is possible to extract an isogeny of degree $2^a - u$: this is achieved via GeneralizedRepresentInteger.

We conclude by remarking that the entire signing procedure requires manipulating ideals to find a response ideal: this mainly involves adding and multiplying ideals, or multiplying an ideal by a scalar. Each of these operations requires computing HNF basis: for example, to multiply two ideals, the four vectors of one ideal are multiplied by the four vectors of the second, thus producing 16 vectors; the HNF computation is thus needed to reduce the number of vectors down to the expected four. Finally, in certain subroutines one needs to compute generators of left ideals. It is known that any left ideal can be generated by two elements, one of which can be chosen to be the norm of ideal. Furthermore, an element σ generates the ideal together with an integer N if the norm of σ divided by N is coprime to N. Thus in the original SQIsign algorithm [2, Algorithm 3.8] one samples elements of the ideal until such a σ is found. It is important to emphasize that the condition on σ is a sufficient condition but it is not a necessary condition. In Section 4 we devise a different condition which is necessary and sufficient (and additive in nature opposed to the previous multiplicative criterion) which will allow us to improve on this previous algorithm.

2.4 Main quaternion subroutines in SQIsign

We now give brief description of the main quaternion subroutines of SQIsign that we refer to or work with in this paper.

Hermite Normal Form and representing ideals. In the SQIsign NIST submission, quaternion lattices and ideals are represented as lattices. More precisely, they are given by a 4×4 integer matrix and a common denominator. These two pieces describe a basis of the lattices, which is a matrix of rationals. For ideals, a pointer to their left order and their norm are stored additionally.

In the first round submission, all matrices describing lattices were furthermore required to be in Hermite Normal Form (HNF) [8, 27]. Even though this requirement was dropped in the second round, HNF computations remain very frequent in SQIsign. On average over 100 runs, key generation and signing together made 6 HNF calls on all 3 NIST levels, in the SQIsign NIST-2 code.

More precisely, SQIsign's reference implementation uses operations such as multiplication and intersection of ideals and other lattices [2, 27]. Since these ideals are represented as lattices given by a basis, 4 generators of both inputs to an operation are known. To obtain the product of two ideals (or lattices) each of the four generators of the first input is multiplied (as a quaternion) with each of the generators of the second input, and as a result a set of 16, generators of a 4-dimensional lattice is obtained. Similarly, computing an intersection involves obtaining a representation of a lattice given by 8 generators. It is then necessary to reduce this large set of generators to a lattice basis. For this, both the first and the second round NIST submissions use a Hermite Normal Form (HNF) computation. For pseudocode of an HNF computation, we refer to [9, Chapter 1.4].

Lattice reduction. In SQIsign, lattice reduction on quaternion ideals and lattices of rank 4 is used in the response generation and the ideal-to-isogeny translation steps, in order to obtain a short basis. In the response step, this is necessary for obtaining a response which can be efficiently represented, and therefore contributes to the small size of the signature. In the ideal-to-isogeny subroutine, it is necessary to find two ideals of short norm, both equivalent to a given ideal I. These are obtained from short norm elements of I. Finding such elements requires lattice reduction in dimension 4, since I is a quaternion ideal.

Traditional lattice reduction algorithms are not constant-time. The recent work [15] proposed a constant-time lattice reduction. Since this algorithm is therefore already studied, we will not investigate it in the following. However, given the implementation in [15] for SQIsign parameters is slow to the point of being impractical for the fully proven iteration counts, and the second round SQIsign submission uses a fast floating-point lattice reduction algorithm, L2 [2, Section 3.1, 4.1], we expect that lattice reduction in constant time will have a major impact on the performance of SQIsign, unless further research improves over the results of [15].

Representing integers. GeneralizedRepresentInteger is an important component in the quaternion arithmetic layer of SQIsign. We provide the pseudocode in Algorithm 1, following [2]. It is responsible for solving norm equations of the form, $x^2 + qy^2 + p(z^2 + qt^2) = M$, where $M \in \mathbb{Z}$ is the target norm and q is a fixed integer parameter determined by the embedding context. This subroutine is invoked in two structurally distinct contexts within the signing protocol, each imposing different requirements on the algorithm.

Specifically, GeneralizedRepresentInteger can be seen to have two versions, depending on whether the quaternion order is the fixed special extremal order $\mathcal{O}_0$ or any another special extremal maximal order $\mathcal{O} \subset B_{p,\infty}$. The special extremal order $\mathcal{O}$ contains the suborder $\mathbb{Z}[\omega] + j \cdot \mathbb{Z}[\omega] \subset \mathcal{O}$, with $\omega \in B_{p,\infty}$ such that $q = -\omega^2$ and j is from the standard basis of $B_{p,\infty}$. When $\mathcal{O} = \mathcal{O}_0$, the order is chosen such that the basis elements satisfy $\omega = i$ and the associated quadratic form becomes $x^2 + y^2 + p(z^2 + t^2)$, corresponding to $q = 1$. This special case arises in RandomIdealGivenNorm [2, Algorithm 3.10], where the goal is to find a uniformly random left ideal of norm M (where M is an odd integer) in $\mathcal{O}_0$. In this case, GeneralizedRepresentInteger solves $x^2 + y^2$ (since, $q = 1$) using Cornacchia's algorithm which requires computing a modular square root, $\sqrt{-1} \mod M$.

In contrast, when RepresentInteger is used inside a subroutine called FixedDegreeIsogeny [2, Algorithm 3.15], it finds quaternion elements in an extremal maximal order $\mathcal{O}$ different from $\mathcal{O}_0$. FixedDegreeIsogeny is used to define isogenies of odd degree u between two supersingular curves. In this case, q is a small positive integer other than 1. The input to Cornacchia in this case requires a modular square root $\sqrt{-q} \pmod M$ to solve the norm equation, $x^2 + qy^2 = M$. In this case, $M = u \cdot (2^{e_{FDI}} - u)$, where e_{FDI} is a fixed scheme parameter.

These two scenarios: fixed $\mathcal{O}_0$ with $q = 1$, and general $\mathcal{O}$ with a small q different from 1, lead to different execution flows. Distinct branches of the modular square root computation are executed depending on the congruence class of M modulo 4 or 8. Moreover, the Euclidean division in Cornacchia's algorithm also adds to data-dependencies within the algorithm. As GeneralizedRepresentInteger appears in both the commitment and response phases, constant-time implementations that support both the cases, that is, when $q = 1$ and $q \neq 1$, are essential.

After computing a suitable set of integers x, y, z, t, the GeneralizedRepresentInteger algorithm constructs a quaternion element γ in a given order $\mathcal{O}$ by embedding the integer coordinates into the internal representation of the order. It then ensures that γ is primitive by dividing out any common integer factors from its coefficients. Finally, an appropriate linear transformation is applied to γ within $\mathcal{O}$, ensuring that the resulting element both belongs to $\mathcal{O}$ and has the desired norm M. With the exception of some GCD computation, most of these operations run in constant time.

3 Hermite Normal Form computation in constant-time

In this paragraph, we discuss the Hermite Normal Form (HNF) computation, with respect to constant time and the requirements of SQIsign. As stated in

Algorithm 1 GeneralizedRepresentInteger$(M, \omega, \mathcal{O}, \mathsf{IsoCond})$ [2]

Input: $M \in \mathbb{Z}$ such that $M > p$, M is also odd.
Input: $\omega \in B_{p,\infty}$, such that $q = -\omega^2$ is a positive integer and $q \equiv 1 \pmod 4$
Input: $\mathcal{O}$ a special extremal maximal order containing the suborder $Z[\omega]+jZ[\omega] \subset O$, with j from the standard basis of $B_{p,\infty}$
Input: IsoCond is a boolean flag only relevant for $-\omega^2 = 1$, enforces a modular condition for FixedDegreeIsogeny [2, Algorithm 3.15].
Output: $\gamma = x + y\omega + zj + t\omega j \in \mathcal{O}$ with $n(\gamma) = M$.

Initialize $q = -\omega^2$, counter $= 0$, bound $= \lceil \frac{4M}{p\sqrt{q}} \rceil$, found $=$ `false`
while not found and counter $<$ bound **do**
  Set counter $=$ counter $+ 1$
  Sample z uniformly from $[1, \ldots, m]$ for $m = \left\lfloor \sqrt{\frac{4M}{p} - q} \right\rfloor$
  Sample t uniformly from $[-m', \ldots, m']$ for $m' = \left\lfloor \sqrt{\frac{4M - pz^2}{pq}} \right\rfloor$
  Set $N = 4M - p(z^2 + qt^2)$
  if N is a prime **then**
    Set res $=$ Cornacchia(q, N)
    Set found $=$ (res $\neq NULL$)
    if found **then**
      Set $x, y =$ res
    if found `AND` IsoCond `AND` $(q == 1)$ **then**
      if $(x \not\equiv t \pmod 2)$ **then**
        Swap(x,y)
      Set found $= (x - t \equiv 2 \pmod 4)$ `AND` $(y - z \equiv 2 \pmod 4)$
    if found **then**
      Set $\gamma = x + y\omega + zj + t\omega j$
      Set d to be the biggest scalar such that $\gamma/d \in \mathcal{O}$
      Set found $= (d == 2)$
    if found **then**
      return γ/d
return `fail`

the preliminaries Section 2, it is a core subroutine of the quaternion module of SQIsign. We first introduce what an HNF is, then explain which of its properties are used in SQIsign. Then we point out that the classical algorithm to compute an HNF is essentially constant-time and give pseudocode for a fully constant-time version. We explain the differences between our HNF and the one used in the SQIsign NIST-implementation. We finally detail some remaining challenges for a more efficient constant-time implementation.

3.1 Definitions and use

Definition 1 (Hermite Normal Form (HNF)). *A full-rank $n \times m$ matrix with integer coefficients is in (column) HNF if:*

– *All columns except the rightmost n are zero: These form a $n \times n$ matrix denoted M'*
– *All coefficients of M' are positive or zero*
– *All diagonal coefficients of M' are positive and non-zero*
– *All non-diagonal coefficients of M are strictly smaller than the diagonal coefficient on the same line*
– *M' is either upper- or lower triangular (we call these variants then a lower- or upper-triangular HNF respectively)*

A matrix with rational coefficients is said to be in HNF if it is given as a HNF of an integer matrix and a minimal positive common denominator of all coefficients.

Each full-rank integer or rational matrix can be transformed into a matrix in HNF by invertible integer linear combinations on the columns. Therefore, the HNF computation preserves the lattice generated by the columns of a matrix and it makes sense to speak of a Hermite Normal Form for a lattice.

Definition 2 (HNF for lattices). *A full-rank lattice is said to be in HNF if it is given by a rational matrix in HNF. Usually, we use only the last 4 columns of that matrix, so that a lattice in HNF is given by a denominator and an integer basis matrix in HNF.*

Every lattice (and therefore also a quaternion ideal) can be represented by a set of linear generators, and therefore every lattice admits a HNF. The HNF representation of a lattice in a fixed basis (we use $(1, i, j, ij)$ for lattices in our quaternion algebra) is unique.

This unicity is the main property of the HNF used in SQIsign, as it is used to test equality of lattices. The positivity and triangularity are not used, even if they are also defining properties of the HNF. The most important property of the HNF is however that it provides a basis which can be computed efficiently from any set of $\mathbb{Z}$-linear generators of the lattice. This property of the HNF computation is used in the lattice- and ideal-addition, multiplication and intersection algorithms in the SQIsign NIST-implementation.

3.2 A HNF computation algorithm

The algorithm for computing a HNF from a matrix representing a lattice is for example given in Cohen's book [9, Algorithm 1.4.8]. It consists in iterating through the rows of the matrix (from bottom to top for the lower- from top to bottom for the upper-triangular HNF), and computing Bézout coefficients and greatest common divisors (gcd) of the coefficients from the diagonal (of the output square matrix) towards the side of the matrix which will be zero (left for upper- right for lower-triangular HNF). Then the Bézout coefficients are used to put the lowest possible positive value (the gcd) on the diagonal in that line by using them as coefficients of linear combinations of the columns. Finally, all other coefficients in that row are reduced modulo that value, again by linear combinations of the columns. In addition, most implementations of

the HNF algorithm use an additional modulus m as input, in order to bound the sizes of the intermediary values in the computation. All linear combinations in the algorithm are then taken modulo m. In order to obtain correct output, m must be a multiple of all diagonal coefficients of the output matrix. Then, the choice of m does not impact the output. For example, (a small multiple of) the determinant of the output matrix or the ideal norm if the input is matrix representing a quaternion ideal are suitable choices.

The resulting algorithm is obviously iterative, with the operations only depending on the dimensions of the input matrix, which in SQIsign is allowed to be known. The intermediary integers are bounded by $2m^2$. The only issue is that one must handle bad inputs to the extended gcd (xgcd) function such as only-zero inputs and ensure this function always produces usable output. In particular, it must output the minimal Bézout coefficients, except that the coefficient of the diagonal must always be non-zero. When these complications are taken care of, one obtains a constant-time HNF algorithm, as given in Algorithm 2. To achieve constant time, the assignments after an if must be implemented as constant-time conditional assignments.

Algorithm 2 computes a lower-triangular HNF, contrary to the SQIsign NIST implementation which uses upper-triangular ones. As only the unicity of the HNF and the fact that it is a basis are used in SQIsign, this does not impact the other quaternion algorithms in the scheme. We made this choice because our constant-time Ideal Generator algorithm from Section 4 requires a lower-triangular HNF. The pseudocode in Algorithm 2 can be easily adapted to compute an upper-triangular HNF by changing the indices of some loops in the following way:

- Line 1: $k = n$
- Line 2 from $i = 3$ downto $i = 0$
- Line 3 $k = k - 1$
- Line 4 $j = k - 1$ downto $j = 0$
- Line 16 $h = i + 1$ up to $h = 3$

3.3 Efficiency concerns

Even if obtaining a constant-time HNF implementation is therefore quite straightforward, some challenges with respect to efficiency remain. First, we expect a large overhead compared to a non-constant-time implementation due to the requirement of constant-time xgcd computations. The HNF computation in SQIsign is usually called on a $4 \times n$ matrix with n equal to 4, 8, or 16. Given the above algorithm, this requires $4(n-2)-2 = 4n-10$ xgcd computations. As these constant-time xgcd have complexity logarithmic in the input size, and the inputs in SQIsign are large, this will heavily impact the runtime. This overhead is not measured in our benchmarks, since we do not use constant-time integer arithmetic yet.

In addition, the large integer sizes occurring during the computation will affect the performance when using a constant-time integer arithmetic. The largest

Algorithm 2 ct-HNF(n, G, m)

Input: G an integer $4 \times n$ matrix with $n \geq 4$ and of full rank
Input: A parameter $m > 0$, multiple of the determinant of a basis of the column span of G. (see Section 3.2 above)
Output: A basis H in HNF of the column span of G.

Set $k = -1$
for $i = 0$ **up to** $i = 3$ **do**
  Set $k = k + 1$
  for $j = k + 1$ **up to** $j = n - 1$ **do**
    Set d, u, v such that

$$d = uG_{k,i} + vG_{j,i} \quad \text{with} \quad d = \gcd(G_{k,i}, G_{j,i}) > 0,$$

    where $u > 0$ and v have minimal absolute value
    Set $c = uG_k + vG_j$
    Set $r_u = G_{k,i}/d$ and $r_v = Gj, i/d$
    if $G_{j,i} == 0$ **then**
      Set $r_u = r_u + 1$
    Set $G_j = r_u G_k - r_v G_j \mod m$
    Set $G_k = c \mod m$
  Set d, u, v such that

$$d = uG_{k,i} + vm \quad \text{with} \quad d = \gcd(G_{k,i}, m) > 0,$$

    where $u > 0$ and v have minimal absolute value
  Set $H_i = uG_k$
  if $H_{i,i} == 0$ **then**
    Set $H_{i,i} = H_{i,i} + 1$
  for $h = i - 1$ **downto** $h = 0$ **do**
    Set $H_h = H_h + \lfloor H_{h,i}/H_{i,i} \rfloor H_i$
  if $G_{k,i} == 0$ **then**
    Set $G_{k,i} = m$
return x

integers encountered at runtime in the SQIsign quaternion module are measured during HNF computations (above 12 000 bits in the NIST-2 implementation[7] against only 5000 bits outside of HNF-related operations). The use of the modulus m in the HNF computation does not really mitigate this: First, since it is most often the determinant of the matrix, it is about 4 times as large as the matrix coefficients. Additionally, the linear combinations before reducing modulo m double again the size of the integers used. This clearly makes computations slower as if smaller integers were used. Additionally, the integers in an HNF computation have very different sizes. Since the output is triangular, many operations are made on zeros, or on the small gcd and final coefficient values. Thus, a constant-time and fixed-size integer implementation is expected to worse

[7] These values are also confirmed by the results in [18].

performance a lot compared to the current variable-size integer implementation using the GMP library.

We therefore expect that using constant-time arithmetic will lead to a major slowdown of the ct-HNF computation and therefore on all operations based on it, including lattice addition, multiplication and intersection.

4 Finding ideal generators in constant time

In the current SQIsign version submitted to the second round of the NIST PQC competition [2, Algorithm 3.8] all left ideals are stored by an integral basis in Hermite Normal Form (which is 4 quaternion elements). However, for certain subroutines it is more convenient to have two generators: a quaternion element and an integer N, that is the norm of the ideal. The current SQIsign version (and as a matter of fact all previous ones) search the ideal for a quaternion α such that $N(\alpha) = N \cdot k$ where k is coprime to N and then it can easily be shown that N and α generate the ideal. This condition can be interpreted as kind of a "multiplicative" condition for generating an ideal and the aforementioned algorithm is clearly not constant-time. In this section, we will take a very different approach and provide an "additive" condition for ideal generation which will make the constant-time algorithm a lot simpler.

Let I be an invertible left ideal of a maximal order $\mathcal{O}$. Our goal is to provide a simple method for finding a generator of I starting from its HNF basis.

Lemma 1. *Let I be a left ideal of $\mathcal{O}$ of norm N. Let $\pi : \mathcal{O} \to \mathcal{O}/N\mathcal{O}$ be the reduction modulo N map which can be restricted to I. Then for $\alpha \in I$ the elements α and N generate I if and only if $\pi(\alpha)$ generates $\pi(I)$ as a left ideal.*

Proof. If α, N are generators, then $\pi(\alpha)$ has to be a generator of $\pi(I)$ as every element in I is an $\mathcal{O}$-linear combination of α and N. Now, let us look in the other direction. Let β, N be generators of I. Then we just have to prove that α, N will generate β. Since β modulo N is in $\pi(I)$ we have that $\beta \equiv o \cdot \alpha \pmod{N}$ which means that there exists $o_1, o_2 \in \mathcal{O}$ such that $\beta - o_1\alpha = No_2$ which proves our claim. □

A left ideal I in $\mathcal{O}$ is called primitive if I is not contained in $l\mathcal{O}$ for any integer $l > 1$. From now on, we will assume that I is primitive as it represents a cyclic isogeny. Let us identify $\mathcal{O}/N\mathcal{O}$ with $M_2(\mathbb{Z}_N)$ (proven inside [31, Theorem 42.1.9]). A primitive left ideal in $M_2(\mathbb{Z}_N)$ is a left ideal which is not contained in $lM_2(\mathbb{Z}_N)$ for any $l > 1$ dividing N. A primitive left ideal clearly reduces modulo N to a primitive left ideal.

Lemma 2. *Every primitive left ideal can be generated by a matrix of the form*

$$\begin{pmatrix} x & y \\ 0 & 0 \end{pmatrix}$$

such that $\gcd(x, y)$ *is coprime with* N.

Proof. Follows from [19, Lemma 7.2]. □

Using the above characterization, we can prove several useful facts about primitive ideals in $M_2(\mathbb{Z}_N)$.

Proposition 1. *Let I be a primitive left ideal in $M_2(\mathbb{Z}_N)$. The following hold:*

1. *I contains an element of trace 1*
2. *If two primitive ideals contain each other, then they are equal*

Proof. Let $\begin{pmatrix} x & y \\ 0 & 0 \end{pmatrix}$ be a generator of I. Since I is primitive, there exists $a, b \in \mathbb{Z}_N$ such that $ax + by \equiv 1 \pmod{N}$. Then observe that

$$\begin{pmatrix} a & 0 \\ b & 0 \end{pmatrix} \cdot \begin{pmatrix} x & y \\ 0 & 0 \end{pmatrix} = \begin{pmatrix} ax & ay \\ bx & by \end{pmatrix}$$

thus its trace is $ax + by = 1$ which proves the first claim.

Now we look at the second claim. Assume that the ideal generated by $\begin{pmatrix} x & y \\ 0 & 0 \end{pmatrix}$ contains $\begin{pmatrix} u & v \\ 0 & 0 \end{pmatrix}$. Every element in the first ideal has the following form

$$\begin{pmatrix} \alpha x & \alpha y \\ \gamma x & \gamma y \end{pmatrix}$$

thus for the containment to hold γx and γy have to be both 0. However, one has that $ax+by = 1$, thus if we multiply this equation by γ we get $0 = a\gamma x+b\gamma y = \gamma$ which implies that $\gamma = 0$. If α is not coprime to N, then the second ideal is not primitive. Otherwise the two ideals are clearly equal as α has an inverse. □

Corollary 1. *Let I be a primitive left ideal of $\mathcal{O}$ of norm N. Let $\alpha \in I$ whose trace is coprime to N. Then $\pi(\alpha)$ generates $\pi(I)$.*

Proof. The ideal generated by $\pi(\alpha)$ is clearly primitive because every element in a non-primitive ideal has trace divisible by some l dividing N. The ideal generated by $\pi(\alpha)$ is clearly contained in $\pi(I)$ and since they are both primitive, Proposition 1 implies that they are equal. □

Theorem 1. *Now assume that we have an HNF basis of a primitive left ideal I where the basis is composed of the elements, $a_1 + a_2i + a_3j + a_4k, a_5i + a_6j + a_7k, a_8j + a_9k, a_{10}k$. Then $a_1 + a_2i + a_3j + a_4k$ will generate I together with N.*

Proof. First, observe that the trace of $\alpha = a_1+a_2i+a_3j+a_4k$ has to be coprime to N. Indeed, assume that $l > 1$ divides both the trace of α and N. Then every element in I would have a trace divisible by l which contradicts Proposition 1. Since $Tr(\alpha)$ is coprime to N, Corollary 1 implies that $\pi(\alpha)$ generates $\pi(I)$. Finally, Lemma 1 implies that α and N generate I. □

Now Theorem 1 provides a very simple algorithm for finding generators of ideals as just one well-chosen basis element of the HNF basis will suffice. As discussed, since SQIsign [2] stores ideals with an HNF basis, one immediately has a constant-time algorithm for generating ideals. Interestingly, this approach is actually faster than the previous "multiplicative" approach as it does not require any nested iteration.

5 Representing integers in constant time

In this section, we describe two constant-time variants of the GeneralizedRepresentInteger algorithm, tailored to the odd integer M it needs to represent, and depending on the value of q (as mentioned in Section 2.4). As discussed previously, the algorithm operates by first randomly sampling two of the four coordinates, z and t, of the quaternion γ, which determines a 'pseudo-norm' $N = 4M - p(z^2 + qt^2)$. The goal is then to find integers x and y which are solutions of the Diophantine equation $x^2 + qy^2 = N$. A standard method for solving such equations is Cornacchia's algorithm, however, three major obstacles prevent a direct use of Cornacchia's algorithm in a constant-time implementation of GeneralizedRepresentInteger:

- The current implementation of GeneralizedRepresentInteger [2, Algorithm 3.12] typically checks if N is prime (demonstrated in Step 7 of Algorithm 1). In a constant-time setting, primality tests are problematic because they are inherently data-dependent and cannot be efficiently implemented without timing leakage. To circumvent this issue, we modify the algorithm to iterate with any value of N until a solution is found, regardless of its primality, albeit restricted to a specific congruence classes modulo 4, as detailed later in the section.
- The second challenge arises in computing a modular square root of $-q$ modulo N in constant time, required for the initialization of Cornacchia's algorithm. The current modular square root algorithm implemented in SQIsign [2, Algorithm 3.1] has conditional branches depending on the residue class of N mod 4 or 8, leading to significant timing variability. To avoid this, we restrict the sampling of z and t such that the resulting N always satisfies $N \equiv 1 \pmod 4$. This in turn, enables us to use either an exponentiation (when $q = 1$, recall from Section 2.4) or a constant-time ct-Tonelli-Shanks algorithm (when q is different from 1, recall from Section 2.4) to compute a modular square root. Moreover, to accelerate this exponentiation and make it constant time, we additionally also ensure that $N \equiv 2 \pmod 3$, allowing us to select a fixed quadratic non-residue modulo N with no conditional branching.
- Cornacchia's algorithm proceeds by executing the Euclidean algorithm to find a solution to the Diophantine equation, $x^2 + qy^2 = N$. However, the classical Euclidean algorithm has data-dependent loop bounds and branches, making its execution non-constant time. We eliminate these conditional

branches by fixing the loop bounds and also replacing the division steps with a regularized algorithm that uses constant-time, branch-free arithmetic operations.

Since both constant-time variants of GeneralizedRepresentInteger rely on a constant-time implementation of Cornacchia's algorithm, we first present this algorithm. We then proceed to describe the complete constant-time constructions of the two variants in the later sub-sections.

5.1 Cornacchia's algorithm in constant time

Cornacchia's algorithm [10] states that, for two co-prime integers N and q, the solution of the Diophantine equation, $x^2 + qy^2 = N$, in coprime integers x and y (if any) is given by the Euclid's algorithm applied to the pair (x_0, N) where x_0 is any root of $x^2 \equiv -q \pmod{N}$. The algorithm stops when in the successive sequence (r_n) of remainders, it finds an r_k satisfying the relation, $r_k^2 < N \leq r_{k-1}^2$, which is equivalent to a half-GCD algorithm. The solution to the algorithm is then $\left(x = r_k, y = \sqrt{(N - r_k^2)/q}\right)$ if y is an integer, otherwise the process is repeated with another modular square root x_0' until all such roots get exhausted. We provide a constant-time version of this in Algorithm 4.

Algorithm 3 ct-IntSqrt(a)

Input: $a \in \mathbb{Z}$ a k-bit integer.
Output: Boolean b indicating whether a is a perfect square, and $s = \sqrt{a}$ if $b = \texttt{true}$.

1: Initialize $s_0 = (1 \ll \lfloor \frac{k}{2} \rfloor)$ ▷ Returns an estimated $\sqrt{a}$ from a's bitsize
2: **for** $i = 1$ to $\log_2 k + 1$ **do**
3: Compute $q, r = \mathsf{Euclidean\_division}(a, s_{i-1})$ ▷ Constant-time version in [23]
4: Set $s_i = (q + s_{i-1})/2$
5: Set $b = (s^2 == a)$ ▷ $s = s_{\log_2(k+1)+1}$
6: **return** b, s

In addition, we also provide a constant-time algorithm (Algorithm 3) for checking if an integer is a square and if so, finding its square root. We provide the proof of correctness and constant-timeness for Algorithm 3 and Algorithm 4 in the following parts. Note that [22] provides a constant-time algorithm for computing square roots up to a certain precision. However, this algorithm is considerably slower than what we provide and also requires to work with integers up to a very large precision. The gain in our algorithm comes from the fact that we don't need to compute exact square roots for arbitrary numbers but only for integers that are squares. The proof of correctness and constant-timeness is given in Section A.1.

Theorem 2. *The algorithm ct-Cornacchia given in Algorithm 4 is correct and terminates in constant time for a given size of the inputs.*

Algorithm 4 $\mathsf{ctCornacchia}(M, u, q, l)$

Input: $M, u, q \in \mathbb{N}$ with $u^2 \equiv -q \pmod{M}$
Input: $l = \lfloor \sqrt{M} \rfloor$
Output: x, y such that $x^2 + q \cdot y^2 = M$

1: Set $k = \mathsf{max}(\mathsf{nbit}(M), \mathsf{nbit}(u))$ ▷ nbit returns the bit-size of the input
2: Set $n = k + 1$
3: Set $a, b = M, u$
4: **while** $n \geq 0$ **do**
5: Set $c = \mathsf{max}(a, b)$
6: Compute $k_c = \mathsf{nbit}(c)$
7: Set sign $= \mathsf{msb}(l - c)$ ▷ Two's complement representation
8: Set $b = \mathsf{min}(a, b) \cdot \mathrm{sign}$
9: Set $a = c - (b) \ll \alpha$ ▷ $\alpha = \alpha'$ if $\alpha' \geq 0$, else $\alpha = 0$
10: Set $n = n - 1$ ▷ $\alpha' = \mathsf{nbit}(c) - \mathsf{nbit}(b) - 1$
11: Set $x = a$
12: Set $y = \mathsf{ct\text{-}IntSqrt}(M - x^2/q)$
13: **return** x, y

Proof. First, we prove correctness. Without loss of any generality, let, $a > b$ such that, $\mathsf{nbit}(a) \geq \mathsf{nbit}(b)$ and hence, $k = \mathsf{nbit}(a)$ in Step 1. The number of iterations of the *while* loop will be $n = k + 1 > 0$. In the first iteration of the algorithm, $c = a$ in Step 5 and $k_c = k$ in Step 6. The boolean variable 'sign' takes the value 0 when $c < l$, otherwise its value is 1. Considering a two's complement representation, this means checking the most significant bit of the value of $l - c$. In constant time, 'sign' can be evaluated as, sign $= ((l - c)\mathtt{AND}(1 \ll k_c)) \gg k_c$. Then, in step 8, if sign $= 1$, then $b = \mathsf{min}(a, b) = b$, otherwise $b = 0$. Let us assume that in this iteration, $c > l$ so that sign $= 1$. In Step 9 the value of a gets updated as, $a \leftarrow a - 2^{\alpha} b$.

First we show that when $\alpha \geq 0$ and $2^{\alpha} b < c$, then $c - 2^{\alpha} b$ is a partial reduction of c by b. An Euclidean division of c by b is equivalent to performing successive subtraction of c by b until the result is between 0 and $c - 1$. Thus, if $\alpha \geq 0$, then $2^{\alpha} b$ is a multiple of b. So computing $c - 2^{\alpha} b$ means merging together successive subtraction of c by b and since $2^{\alpha} b < c$, which means we compute a partial reduction.

Now we show that computing successive subtractions of c by $2^{\alpha} b$ follows the algorithmic flow of Euclidean division. Indeed, that computing $c \leftarrow c - 2^{\alpha} b$ means computing a partial reduction of c by b. Since we repeat this operation until $c_i < c_0$ ($c_i = \mathsf{max}(a, b)$ in the i-th iteration), hence this equates to the modular reduction of c by b. This is exactly the operation performed in the Euclidean algorithm, the only difference is that we have decomposed the modular reduction into multiple partial reduction to facilitate a constant-time flow. Finally, we prove that starting from $n = k + 1$ and assuming a is a k-bit integer, when $n = 0$, $a_i = \mathsf{ctCornacchia}(a, b)$.

We have shown that Algorithm 4 follows the same flow as the Euclidean algorithm. To obtain the first output, we need to perform a GCD but stop "midway" (half-GCD algorithm) which is ensured by the variable 'sign' in Step 7. 'sign' is computed as sign $=$ msb$(l - c_i)$, in two's complement representation where $l = \sqrt{a}$. Thus, when $c_i > l$, sign $= 1$. Consequently, following a partial reduction, when $c_i < l$ then 'sign' will be set to zero, thereby preventing further subtractions in Step 8 and 9. Thus, the sequence of $(a_i)_i$ becomes 'stagnant' at this specific value of c_i, which is the output we want.

Note that in Step 10, we introduce the variable α, defined as $\alpha =$ nbit$(c) -$ nbit$(b) - 1$ or 0 if the previous value is negative. The variable α serves two purposes. First, α aids in a decent estimate for the total number of loops required for correct termination, which would be highly inefficient based only on simple successive subtractions as the required number of loops would be quite large. Second, α also allows to set an upper bound for the required number of loops for Algorithm 4.

The variable α mimics the Euclidean division, $a/b = c - q_{ot}b$, by merging successive subtractions together (left-shifting b by α). To merge the maximum number of subtractions together and perform a partial reduction, we set $\alpha =$ nbit$(c) -$ nbit$(b) - 1 \geq 0$ else $\alpha = 0$, thereby ensuring that the value of $b \ll \alpha$ will be one-bit lesser than c, which in turn guarantees the correctness of our algorithm. Clearly, an interesting consequence is that it also provides us with an upper bound for n (Step 4). Since $b \ll \alpha$ is always 1-bit lesser than c, this means that in the worst case scenario, it will take a maximum of two iterations of n for c to decrease by one bit. The algorithm only requires halving the input a of size k-bits and since we reduce 1-bit per two iterations, this means that the ct-Cornacchia will be computed in k steps, thus our choice of $n = k + 1$ is sufficient for correct termination.

Next, we prove that Algorithm 4 is constant-time. Steps $1, 2, 3, 5, 6, 8, 10, 12$ are constant-time either by constant-time assumption of basic arithmetic operations or by simple constant-time implementation of functions such as maximum, minimum or nbit. Again, the computation of sign$(l-c) = ((l - c)$AND$(1 \ll k_c)) \gg k_c$ is constant-time by the constant-time assumption of basic arithmetic functions. A similar argument for constant-time can be applied to compute α. First, we compute $\alpha' =$ nbit$(c) -$ nbit$(b) - 1$ and define r as the value obtained from performing an AND operation between all the bits of α'. Then, $\alpha = \overline{r} \cdot \alpha'$. Finally, the *while* loop runs a fixed number of times $n = k + 1$ where k is determined based on the maximum size of the inputs that Cornacchia takes in SQIsign's signing algorithm. Thus, for a given fixed input-size, the number of execution steps in Algorithm 4 is independent of any input characteristics and is therefore constant-time. □

5.2 Constant-time GeneralizedRepresentInteger in $\mathcal{O}_0$

We previously identified three obstacles to achieving a constant-time implementation of the GeneralizedRepresentInteger algorithm. In Section 5.1, we addressed

the issue associated with Cornacchia's algorithm. We now focus on the setting on the special maximal extremal order $\mathcal{O}_0$, where $q = 1$ and $\omega = i$. In this case, the challenge of computing a modular square root of $-q$ reduces to computing a modular square root of -1, which has simpler constant-time solutions.

In Algorithm 1, the only remaining non-constant-time step is the primality test in Step 7, which ensures that the pseudo-norm N is a prime number. The primality test guarantees both the existence of a modular square root of -1 mod N and the success of the Cornacchia's algorithm, that requires a modular square root to start with. To make the algorithm constant-time, we remove the primality test and instead add a simple test at the end of our algorithm that checks if the solution generates a quaternion with the correct norm. If the quaternion is indeed of the right norm, then in that case, we use a constant-time conditional-move (ct-cmov) to save and output the solution.

We also change how the coordinates z and t are generated. In the original algorithm, z and t were uniformly sampled from the interval $[1, \lfloor\sqrt{\frac{4M}{p} - q}\rfloor]$ (Steps 4, 5 in Algorithm 1). In the constant-time variant, we replace the randomized sampling by two deterministic sequences $(z_k)_{k\in\mathbb{N}}$ and $(t_k)_{k\in\mathbb{N}}$. The algorithm then iterates, up to a fixed number of iterations, over all possible pseudo-norms N of the form $N = 4M - p(z_k^2 + t_k^2)$, that satisfies the congruence conditions $N \equiv 2 \pmod 3$ and $N \equiv 1 \pmod 4$. This iteration bound (details in Section 5.4) is set such that the algorithm finds a prime N satisfying these congruence conditions. These conditions on a prime N yield important properties: since $N \equiv 1 \pmod 4$, there exists a deterministic square root of -1 mod N. Moreover, by quadratic reciprocity, having $N \equiv 2 \pmod 3$ ensures that 3 is a non-quadratic residue modulo N. Hence, we can express the square root of -1 mod N as: $x = \sqrt{-1} \bmod N = 3^{(N-1)/4} \bmod N$ and hence, we can use a rapid constant-time modular exponentiation in the implementation.

In SQIsign, the GeneralizedRepresentInteger algorithm is used in RandomIdealGivenNorm [2, Algorithm 3.10] (Section 2.4), where the input N is always odd. Thus, for SQIsign primes $p \equiv 3 \pmod 4$ at levels 1, 3 and 5, the congruence constraints on N can be enforced explicitly by constructing sequences of the form $z_k = 12k$, $t_k = 12k + 1$, which translate to $z_k^2 \equiv 0 \pmod 4$ and $t_k^2 \equiv 1 \pmod 4$, and therefore $N = 4 \cdot M - p(z^2 + t^2) \equiv 0 - 3(0^2 + 1^2) \bmod 4 \equiv 1 \pmod 4$. Additional corrections via a "look-alike" Lagrange interpolation ensure the required class modulo 3, which needs the introduction of the variable $d = M \bmod 12$ to work. Explicit formulas for z_k Equation (1) and t_k Equation (2) for SQIsign levels 1 and 3 are

$$z_k(d,k) = 12k + 8\frac{(d-0)\cdot(d-2)}{(1-0)\cdot(1-2)} + 72 \cdot 12 = 12k - 8d \cdot (d-2) + 864, \quad (1)$$

and,

$$t_k(d,k) = 12k + 1 + 8\frac{(d-0)\cdot(d-1)}{(2-0)\cdot(2-1)} = 12k + 4d \cdot (d-1) + 1. \quad (2)$$

For SQIsign level 5, a different sequence is required since the underlying prime satisfies $p \equiv 2 \pmod 3$, in contrast to levels 1 and 3 where $p \equiv 1 \pmod 3$.

Together, these congruence constraints, fixed deterministic sampling and a sufficiently large fixed iteration bound guarantee that GeneralizedRepresentInteger algorithm always succeeds in constant time.

We provide the full constant-time algorithm for GeneralizedRepresentInteger in $\mathcal{O}_0$ in Algorithm 5 and name it ct-RepresentInteger$_{\mathcal{O}_0}$ for brevity.

Algorithm 5 ct-RepresentInteger$_{\mathcal{O}_0}(M)$

Input: $M \in \mathbb{Z}$ such that $M > 2^{20} \cdot p$ and M odd.
Output: $\gamma \in \mathcal{O}_0$ with $n(\gamma) = M$
Output: found, a boolean indicating whether a solution was found

Set $(x_f, y_f, z_f, t_f) = (0, 0, 0, 0)$, found $= 0$
Set $d = M \bmod 12$
for $k = 1$ to l **do** ▷ l is discussed in Section 5.4
  Set $z = z_k(d, k)$ ▷ See Equation (1)
  Set $t = t_k(d, k)$ ▷ See Equation (2)
  Set $N = 4 \cdot M - p \cdot (z^2 + t^2)$
  Set $n = (N - 1)//4$
  Compute $u_0 = 3^n \bmod N$ ▷ Constant-time modular exponentiation
  Compute $x = \mathsf{ctCornacchia}(N, u_0, 1, \sqrt{N})$
  Compute $y = \sqrt{N - x^2}$
  Compute $\gamma =$ make-quaternion-primitive$(x, y, z, t, \mathcal{O}_0)$
  Compute diff $= M - (\gamma[0]^2 + \gamma[1]^2 + p(\gamma[2]^2 + \gamma[3]^2))$
  Set $s = \mathsf{ctIsZero}(\text{diff})$ ▷ See Section 2
  Set $(x_f, y_f, z_f, t_f) = (\gamma[0], \gamma[1], \gamma[2], \gamma[3])$ ▷ Done using $s = 1$ in ctCondMov
  found $= \mathsf{ctCondMov}(1, s, s)$
return found, $\gamma = x_f + y_f i + z_f j + t_f k \in \mathcal{O}_0$

5.3 Constant-time GeneralizedRepresentInteger in $\mathcal{O}$

We now consider the constant-time implementation of Generalized RepresentInteger in the case of a maximal order $\mathcal{O}$ other than $\mathcal{O}_0$. This setting shares similarities with the special case discussed in Section 5.2 but is used in FixedDegreeIsogeny [2, Algorithm 3.15] rather than in RandomIdealGivenNorm [2, Algorithm 3.10]. Two differences are crucial for our constant-time analysis.

First, here, the input M has the special form $M = u \cdot (2^{e_{FDI}} - u)$, where u is an odd integer and e_{FDI} is a fixed scheme parameter. Since u is odd and $2^{e_{FDI}} > 8$, this implies that $M \equiv 7 \pmod 8$. Second, the parameter q is selected from a fixed set of small integers [2, Appendix-B], all congruent to 1 mod 4. Consequently, this version of the algorithm requires computing a square root of $-q \bmod N, q \neq 1$ in constant-time, which is more challenging than the special case of $q = 1$. We will first discuss how to compute the modular square root

of $-q$ in constant time and then our approach of constructing the constant-time algorithm using the two sequences $(z_k)_{k\in\mathbb{N}}$ and $(t_k)_{k\in\mathbb{N}}$.

Constant-time Tonelli-Shanks. The modular square root algorithm used in SQIsign [2, Algorithm 3.1] has multiple branches depending on the input M, it invokes the Tonelli–Shanks algorithm in one of the branches. To obtain a constant-time implementation, all branches in Tonelli-Shanks and in the modular square root computation must be removed.

Our approach builds upon the Tonelli-Shanks variant described in [28]. Specifically, we again use the flexibility in the definition of the pseudo-norm $N = 4M - p(z^2 + qt^2)$, to enforce suitable congruence conditions by carefully constructing z and t. This allows both efficiency improvement and also gets rid of non-constant-time branching in the algorithm. We enforce the following on the pseudo-norm condition: $N \equiv 2 \pmod 3$, $N \equiv 5 \pmod 8$.

The first condition ensures that 3 is a quadratic non-residue modulo N by quadratic reciprocity, while the second condition guarantees that the 2-adic valuation of $N-1$ is exactly two because $N \equiv 5 \pmod 8$. Together, these conditions allow the execution of a Tonelli-Shanks variant in a constant-time manner for all admissible inputs (see Algorithm 6).

Algorithm 6 ct-Tonelli-Shanks(n, N)

Input: An odd prime $N \equiv 5 \pmod 8, N \equiv 2 \pmod 3$ and a quadratic residue n mod N
Output: x such that $x^2 \equiv n \pmod N$

Set $e = 2$, $h = (N-1)/2^e$
Set $z = 3^h$
Compute $y = n^{(N-2^e-1)/2^{e+1}} \mod N$
Set $s = n \cdot y \bmod N$
Set $t = s \cdot y \bmod N$
for $k = e$ down to 1 **do**
    $b = t$
    for $i = 1$ to $k-1$ **do**
        Set $b = b^2 \bmod N$
    Set $s =$ ctCondMov$(s, s \cdot z, b)$
    Set $z = z^2 \bmod N$
    Set $t =$ ctCondMov$(t, t \cdot z \bmod N, b)$
Set $x = s$
return x

Deterministic generation of (z_k) and (t_k). To obtain the above congruence conditions on N, we replace randomized sampling of z and t by deterministic sequences $(z_k)_{k\in\mathbb{N}}$ and $(t_k)_{k\in\mathbb{N}}$, constructed analogously to Section 5.2, but with

additional dependence on $q_3 = q \bmod 3$ and $q_8 = q \bmod 8$. Since any integer congruent to 5 mod 8 is also congruent to 1 mod 4, the sequences are built following the same logic as in Section 5.2, with the addition of a "Lagrange interpolation" correction term on z_k to enforce the desired congruence modulo 8. This approach keeps the sequences simple while ensuring correctness modulo 8.

Let Q denote the set of all values of (q) for a given SQIsign level (see [2, Table p106]), for example, $Q = \{5, 17, 37, 41, 53, 97\}$ for level 1, and, $Q = \{5, 13, 17, 41, 73, 89, 97\}$ for level 3. For each $q \in Q$, we construct sequences $z_k(d_3, q_3, q_8, k)$ (Equation (3)) and $t_k(d_3, k)$ (Equation (4)) for Algorithm 9 where $d_3 = M \bmod 3$, designed to guarantee that the resulting pseudo-norm satisfies $N \equiv 2 \pmod 3$, $N \equiv 5 \pmod 8$.

$$z_k(d_3, q_3, q_8, k) = 12k + 3 \cdot (q_8 - 1)/2 + 2(d_3 - 1)(d_3 - 2) + 4d_3(d_3 - 2)(q_3 - 2) \quad (3)$$

$$t_k(d_3, k) = 12k + 9 - 4 \cdot d_3 \cdot (d_3 - 2) \quad (4)$$

As in the $\mathcal{O}_0$ case, we remove the primality test from Step 7 of Algorithm 1 and replaces it with a simple constant-time test. Moreover, the congruence constraints, fixed deterministic sampling and a sufficiently large fixed iteration bound guarantee that GeneralizedRepresentInteger algorithm always succeeds in constant-time. We provide the full constant-time version in Algorithm 9 in Section A.2 and name it ct-RepresentInteger$_{\mathcal{O}}$ for brevity.

5.4 Computing a bound for iterations of ct-RepresentInteger

Let M be an m-bit integer, according to the prime number theorem, the number of primes less than M is approximately given by $\pi(M) \sim \frac{M}{\log M}$. In the sequences $(z_k)_{k\in\mathbb{N}}$ and $(t_k)_{k\in\mathbb{N}}$, we sample over a subset of $\mathbb{N}$ which all come from certain residue classes. Thus when applying Cornacchia's algorithm after choosing z, t we can assume that probability of the number to represented being prime is $O(1/m)$. Since the goal of ct-RepresentInteger is to find a prime pseudo-norm satisfying the conditions required for Cornacchia's algorithm, the number of iterations must be chosen sufficiently large to ensure, with high probability (i.e., failure probability $s < 2^{-80}$), that at least one prime is encountered during the sampling process which can be written as $x^2 + qy^2$. In general the density of primes (amongst all primes) that can be written in the form $x^2 + qy^2$ is $\frac{1}{2h(-q)}$ [11, Theorem 9.12] where $h(-q)$ denotes the class number of $Z[\sqrt{-q}]$. Thus we need to hit $O(\sqrt{q})$ primes (for all of the given q, these quantities can be calculated precisely) in order to be able to have a solution to the Cornacchia equation. However, since these q are small, one can explicitly calculate these class numbers which in our case are $2, 4, 2, 8, 6, 4$. Thus the worst case requires a 8x iteration count compared to the $q = 1$ case. This could easily be enhanced by only selecting certain residue classes. SQIsign [2] did not consider this as a potential optimization so we will not assume this here. In order to get a proper iteration count we need to estimate

the probability of hitting one prime and then the iteration count has to be multiplied by the appropriate class number. Since we only consider odd integer the probability of hitting a prime is: $P = \frac{\pi(M)}{M/2} \sim \frac{2}{\log M}$.

We model the process as a sequence of independent Bernoulli trials with success probability $P = 2/\log M$, the probability of finding at least one prime in l trials is thus $1-(1-P)^l$. We want to ensure that this probability is above $1-s$, which implies $l > \log s/\log(1-P)$. Substituting $P = \frac{2}{\log M}$, $s = 2^{-80}$ and considering M as a 285-bit integer, we obtain:

$$l > (\log 2^{-80})/\log\left(1 - \frac{2}{\log 2^{285}}\right) \sim 5500.$$

Therefore, to have a failure probability below 2^{-80}, we need at least $l \geq 5500$ iterations when $q = 1$.

An alternative approach: rejection sampling. We presented a constant-time version of the RepresentInteger algorithm, which can be used as a drop-in replacement for the current RepresentInteger algorithm in SQIsign. However, it may be possible to achieve resistance to timing attacks through a different approach that requires modifying the SQIsign protocol and introducing some trade-offs. While this approach is outside of the scope of this work (our goal is to provide constant-time algorithms that can be used within SQIsign, *as it is now*), we give a brief sketch.

The RepresentInteger algorithm is mainly used in three routines: to sample a secret key, to sample a commitment isogeny, and to generate the auxiliary isogeny in the response. In the case of the secret and commitment isogeny, the inputs to RepresentInteger are secret: these are the values u and v in the ideal-to-isogeny algorithm, and they leak non-trivial information about the secret key and the commitment isogeny. However, it may be possible to choose a significantly lower bound for the number of iterations; this change would increase the probability that the algorithm fails, but the SQIsign protocol could simply abort the key generation or signing procedure and restart with fresh randomness. This would, overall, drastically decrease the number of iterations and the running time, but would also introduce a bias in the distribution of public keys and commitment curves. Such biases make the security proof of SQIsign more complicated [3, Section 7], unless it relies on very ad-hoc assumptions. This thus introduces a trade-off between a more efficient constant-time RepresentInteger algorithm and a cleaner security proof.

In the case of the auxiliary isogeny computation, the situation is different: the input to RepresentInteger is public since it depends on the degree of the auxiliary isogeny, which is revealed as part of the signature. The output of RepresentInteger should still remain secret, because it would otherwise reveal non-trivial information on the commitment isogeny: namely, the output of RepresentInteger gives an isogeny from E_0, which is the pullback of the auxiliary isogeny under the commitment isogeny. This means that it is important to implement RepresentInteger in constant time to prevent leakages, but its iteration count does not

need to be fixed: it can simply loop until the algorithm succeeds and end there. The repetition count depends uniquely on its input (the output depends on the randomness that is sampled fresh every iteration), and thus does not leak any private information.

6 Qlapoti: norm equation solving for ideal-to-isogeny translation

Qlapoti [6] is a recent improvement of the ideal-to-isogeny translation for SQIsign. It is not used in the SQIsign NIST implementation since it only appeared later. It is intended to replace a complex and very memory-consuming subroutine of the NIST submission called find_uv. This subroutine is the only place in the current NIST submission using RepresentInteger on orders different from $\mathcal{O}_0$, and seemed exceedingly unstable (since many improvements seemed imaginable) and hard to make fully constant-time. In particular, future SQIsign versions which use Qlapoti will only need RepresentInteger for $q = 1$, unlike the NIST round 2 submission.

Qlapoti improves on many of these aspects: in particular, it has a low memory footprint and uses only the order $\mathcal{O}_0$. However, it is still a fairly complex search-based algorithm. Its main innovation is the first step, which solves a specific Diophantine equation, which is a norm equation in a quaternion ideal. This step is followed by the ideal-side operations of the ideal-to-isogeny translation, which consists mostly in preparing some torsion basis and finally computing a $(2, 2)$-isogeny chain, before finding the appropriate output curve in the final product using some pairing. Since elliptic curve point manipulations and even pairings are more common in cryptographic implementations than quaternions, we focus in the following on the norm equation resolution.

6.1 A constant-time algorithm for solving Qlapoti norm equations

The norm equation solving part of Qlapoti takes place fully on the quaternion side of SQIsign, and resembles RepresentInteger to some extent. It also crucially relies on Cornacchia's algorithm for decomposition in sum of two squares, and also enumerates through many possible inputs to Cornacchia until a suitable one is found. The main difference is that Qlapoti requires more other operations, in particular lattice reduction in two and four dimensions and random sampling of small ideal generators (or elements).

Similarly to our approach for RepresentInteger, we will pursue a relatively naive approach of iterating sufficiently to make sure we find a solution. As for RepresentInteger, this requires lots of adjustments inside the algorithm in order to get error-free execution and correct output without conditional branches. The resulting pseudocode is given in Algorithm 7.

Overview on the subroutines. While most of the operation in Algorithm 7 are just simple integer, linear or quaternion arithmetic, some subroutines require more care:

Algorithm 7 ct-Qlapoti(J)

Input: Left $\mathcal{O}_0$-ideal J.
Output: Equivalent ideals I_1 and I_2 with $n(I_1) + n(I_2) = 2^e$.

1: Compute $B = \mathsf{ct\text{-}LLL}(J)$
2: Compute $I = \chi(B_0, J)$
3: Compute $n_I = n(I)$
4: Compute $B = \mathsf{ct\text{-}LLL}(I)$
5: Set res $= \perp$
6: **for** $i = 1$ to *bound* **do** ▷ *bound* is the loop iteration number discussed below
7: Set $\alpha = \mathsf{ct\text{-}SmallElement}(B)$
8: Set $r = n(\alpha)/n_I$
9: Denote a_α and b_α the first two coordinates of α in basis $1, i, j, ij$
10: Set bad $= (\gcd(2a_\alpha, n_I) \neq 1$ `AND` $\gcd(2b_\alpha, n_I) \neq 1)$
11: ct-compute shortest (s, t) such that $2a_\alpha s + 2b_\alpha t = 2^e - 2r \mod n$;
12: Set $z = 2(2^e - 2r - 2a_\alpha s - 2b_\alpha t)/n_I - s^2 - t^2$;
13:

$$\text{Set bad} = \text{bad } \texttt{OR} \begin{cases} z < 0, \texttt{OR} \\ z \equiv 0 \pmod 4 \ \texttt{AND} \ (\texttt{NOT}(s \equiv t \equiv 0 \pmod 2)), \texttt{OR} \\ z \equiv 1 \pmod 4 \ \texttt{AND} \ (s \equiv t \pmod 2), \texttt{OR} \\ z \equiv 2 \pmod 4 \ \texttt{AND} \ (\texttt{NOT}(s \equiv t \equiv 1 \pmod 2)), \texttt{OR} \\ z \equiv 3 \pmod 4. \end{cases}$$

14: Set sol $= \mathsf{ctCornacchia}(z)$;
15: ct-Swap(res, $sol|s|t$, (sol $\neq \perp$) `AND` (`NOT` bad))
16: ct-Swap($sol|s|t$, res, (res $\neq \perp$))
17: Set $z_0, z_1 =$ sol
18: ct-Swap($z_0, z_1, z_0 \not\equiv s \pmod 2$)
19: Set $a_1 = (z_0 + s)/2$, and $b_1 = (z_1 + t)/2$
20: Set $a_2 = s - a_1$, and $b_2 = t - b_1$
21: Compute $\beta_1 = n_I(a_1 + b_1 i) + \alpha$, and $\beta_2 = n_I(a_2 + b_2 i) + \alpha$
22: Compute $I_1 = \chi_I(\beta_1)$, and $I_2 = \chi_I(\beta_2)$
23: **return** I_1, I_2

– Step 1 and Step 4: This is just Lattice reduction in dimension four, as provided by [15]
– Step 7: Sample a small random element of I. Details discussed below.
– Step 11: Relies on computing a short vector in a two-dimensional lattice, which can be done using the constant-time Lagrange algorithm from [15]. The close vector finding is then done using simple linear algebra, just as in the original Qlapoti.
– Step 14: Qlapoti originally uses trial division up to 500 in practice, but its theoretical analysis only assumes Cornacchia to work on primes which are 1 mod 4. Restricting Cornacchia to prime inputs also works experimentally. We can therefore use our ctCornacchia algorithm from above works, and as in RepresentInteger, it does not require any prior primality testing.

Sampling small ideal elements for ct-SmallElement at Step 7 of ct-Qlapoti. The original Qlapoti algorithm uses random generators instead of random small elements here. We therefore have two challenges:

- Find a constant-time algorithm for finding these small elements
- Justify that we can use small elements instead of small generators (and take that change into account in the run time)

For the algorithm, we can use Algorithm 8, which is constant time since b is fixed independently from the input B.

Algorithm 8 ct-SmallElement$_b(B)$

Input: $B = (B_1, B_2, B_3, B_4)$ reduced basis of an ideal I, and a constant bound b on the coefficients of the linear combination to be taken
Output: x random element of I with norm at most $4b^2 n(B_4)$ where $n(B_4)$ is the norm of the largest element in B.

1: Sample t, x, y, z independently uniformly at random in the interval $[-b, b]$;
2: Compute $\alpha = tB_1 + xB_2 + yB_3 + zB_4$;
3: **return** α;

For the replacement of small generators by small ideals, one may notice that the test in Step 10 enforces that all elements considered not "bad" (and therefore susceptible to contribute to the output) are generators, since it enforces a trace coprime to the ideal norm (see Corollary 1). Therefore, one only needs to prevent the αs which are not generators to throw errors in the dummy loops in which they occur. This is something a constant-time code should handle automatically, since it behaves identically whatever inputs it gets (even on invalid ones). The impact of this change on the run time will be studied in the next paragraph.

Loop iteration number. The number of loop iterations required for a negligible failure probability can be derived from the analysis in the Qlapoti paper. The use of ctCornacchia which works only for primes does not impact the number of loops needed, since the analysis in the Qlapoti paper already takes this into account. Furthermore, that analysis counts the loops for α generators, but at same time assumes that their trace is uniformly random when considering its coprimality with the ideal norm. Therefore, our small elements should satisfy this condition just as well (or just as badly) as the generators in the Qlapoti paper, and therefore the same analysis applies. The number of iterations required is therefore the number c reported in [6, Table 3].

6.2 Qlapoti implementation (not in the NIST-2 SQIsign code)

Proof-of-concept C implementation of Qlapoti. Separated from our effort to implement constant-time quaternions for SQIsign, we also implemented the

Qlapoti norm equation solving step in constant time. However, we did not make all subroutines constant-time, in particular, we left the Cornacchia for primes and the lattice reduction as they were given in SQIsign. Lagrange reduction and integer arithmetic are equally not made constant-time. The resulting code does not match exactly the pseudocode above, since the Qlapoti C-implementation already departed from the Qlapoti-pseudocode in order to accommodate the impossibility to compute diagonal (2,2)-isogenies using the x-only arithmetic of the SQIsign NIST implementation. However, it only differs from the above by enforcing some additional conditions and testing for several variants of some potential solutions. This difference (which rejects some results which would be accepted in plain Qlapoti norm equation solving), also creates a need for slightly larger loop iteration bounds.

The resulting implementation seems correct, but is fairly inefficient. Ideal-to-isogeny computations and even full SQIsign signatures can be run using it and pass tests. However, solving a single norm equation takes about 9 seconds at NIST level 1, and hundreds of seconds at higher levels.

Possible improvements to ct-Qlapoti. We emphasize that our algorithm is not optimized, and it relies on a naive approach to constant-time Qlapoti, whose main purpose is to illustrate that all important subroutines are now feasible in constant time. The implementation is even slower since it needs to impose the additional constraints due to the lack of diagonal 2,2-steps in the C code. Improvements to the algorithm are possible, some even fairly simple. For example, the number of iterations of the main loop can be improved by repeating the sampling of α a few times and retaining one which passed the test in Step 10.

While Qlapoti is much more constant-time friendly than the ideal-to-isogeny implementation in the round 2 NIST submission of SQIsign, it is useful to mention that the search-based norm equation solving approach is still not very efficient when made constant-time naively. In case future improvements or new ideal-to-isogeny algorithms are developed (as it has been the case every few months in the past two years), we recommend to keep this criterion in mind when comparing them.

7 Implementation results

The implementation of the proposed constant-time quaternion functions is written in C and is integrated into the SQIsign NIST Round 2 official code, without making any changes to the GMP-based big-integer arithmetic. Our code treats the big-integer arithmetic as a black box and any alternative constant-time library such as from [23] could be substituted without affecting correctness. The implemented quaternion subroutines adhere to strict constant-time coding practices: loops are executed with fixed iteration counts, conditional behavior is expressed through masked selects or constant-time swaps, and no secret data influences either control flow or memory access patterns. Our objective was not to optimize performance but to provide formally constant-time formulations of

these critical subroutines. We view these implementations as a baseline that can be further optimized, e.g. through hand-tuned assembly or vectorization, while preserving constant-time guarantees.

Constant-time evaluation. Beyond the algorithmic arguments given earlier, we validated the constant-time nature of our C implementations experimentally. We employed TIMECOP [1], which automatically tracks the propagation of secret-dependent values and reports whether they affect timing-relevant operations such as branches or memory accesses. In addition, we measured runtimes across actual SQIsign inputs with varying sizes (260, 350 and 500 bits for lvl1, 3 and 5 resp.) whose details we give in the following paragraph.

Performance analysis and limitations. We provide timing results for the proposed constant-time algorithms in Table 1 and the key generation, signature sub-routines in Table 2. All timing measurements were performed on a LENOVO MT 20X1 BU Think FM ThinkPad L14 Gen 2 laptop. The machine is equipped with an 11th Gen Intel(R) Core(TM) i5-1135G7 CPU running at 2.40 GHz, based on the Intel x86-64 (64-bit) architecture, and 16 GiB of memory. The experiments were conducted under Ubuntu 24.10 (Linux-based), with both Turbo Boost and Hyperthreading disabled to ensure stable timings. Table 1 summarizes the performance of our constant-time implementations relative to their non-constant-time counterparts. The constant-time ideal generator

Table 1: Benchmark results for IdealGenerator, HNF, and RepresentInteger.

Function	Version	Median (CPU cycles)			Average (CPU cycles)		
		lvl 1	lvl 3	lvl 5	lvl 1	lvl 3	lvl 5
IdealGenerator	non-ct	28 980	32 112	30 092	60 326	63 646	72 896
	ct	1567	1588	1573	1579	1654	1622
HNF	non-ct	286 757	317 063	352 199	290 043	331 192	387 030
	ct	543 034	546 829	554 583	542 698	546 304	555 377
RepresentInteger	non-ct	66 047 343	77 561 202	159 838 719	97 571 124	145 479 164	211 963 034
RepresentInteger $q = 1, l = 300$	ct	80 118 223	137 064 247	257 256 202	80 940 708	138 613 351	260 160 987
RepresentInteger $q = 1, l = 6000$	ct	1 803 208 021	3 122 012 985	5 836 999 077	1 840 203 914	3 189 369 482	5 962 446 500
RepresentInteger $q = 5$	non-ct	131 426 966	205 108 076	255 437 093	247 064 890	311 030 797	368 036 369
RepresentInteger $q = 5, l = 300 \cdot q$	ct	972 900 769	1 235 153 780	-	993 622 687	1 247 785 628	-

function achieves a 30× speedup, whereas both ct-HNF and ct-RepresentInteger exhibit substantial slowdowns. These numbers, however, do not yet reflect the overhead of fully constant-time big-integer arithmetic. For instance, [23] reports that constant-time GCD can be up to 5000× slower than the GMP implementation for 12000-bit inputs. Since HNF relies heavily on repeated GCD computations, the practical slowdown of ct-HNF will naturally be significantly higher once constant-time GCD and other big-integer routines are integrated.

Analysis of the run-times of ct-RepresentInteger is more involved, as it depends critically on the choice of the iteration bound l that controls the number of iterations of the internal loop (the value l in Algorithm 5 and explained in Section 5.4). A larger value of l reduces the failure probability of the algorithm but it simultaneously increases the number of loop executions and hence the overall running time. In this sense, l is inversely related to the failure probability (though not linearly), but directly proportional to the computational cost. Consequently, setting l introduces a trade-off between runtime efficiency and robustness against failure. Thus, we provide runtimes of ct-RepresentInteger$_{\mathcal{O}_0}$ by setting an experimental bound of $l = 300$ and the theoretical worst case bound of $l = 5000$. We scale l accordingly for ct-RepresentInteger$_{\mathcal{O}}$ (as in Section 5.4). We also notice that the performance of ct-RepresentInteger$_{\mathcal{O}_0}$ is reasonably close to the average runtime of the non constant-time version. This behavior arises from the non constant-time algorithm's stopping condition: it terminates as soon as a solution is found, which yields a much lower median runtime but also includes corner cases where significantly more iterations are required, thereby inflating the average. In contrast, the constant-time version fixes an iteration bound to guarantee a solution in the worst case, and always executes that bound regardless of early solutions. As a result, its runtime aligns more closely with the average behavior of the non-constant-time algorithm, while avoiding its variability. Finally, we note that the performance overhead of both versions of ct-RepresentInteger will also be affected by using constant-time big-integer arithmetic.

Table 2: Benchmarking results for key generation and signing at levels 1 and 3.

Level	Version	Operation	Average Time	CPU Cycles
lvl 1	non-ct	Keygen	118.63 ms	287 124 593
		Signature	264.06 ms	639 138 363
	ct	Keygen	1397.21 ms	3 389 656 327
		Signature	3818.20 ms	9 239 116 335
lvl 3	non-ct	Keygen	273.23 ms	661 187 935
		Signature	593.59 ms	1 436 525 133
	ct	Keygen	2859.86 ms	6 920 319 951
		Signature	8164.68 ms	19 756 491 019

Table 2 reports the runtime of key generation and signature operations using our constant-time subroutines; verification remains unaffected by our proposed changes. It is important to note that these slowdowns arise solely from the constant-time algorithms introduced in this work and do not include the impact of constant-time integer arithmetic or constant-time lattice reduction [15]. Hence, the reported numbers should not be interpreted as the final performance of a fully constant-time SQIsign implementation, but rather as a reference point indicating the relative impact of our subroutines. These results can serve as a guide to estimate the overall slowdown introduced by constant-time building blocks and highlight the need for future work on designing algorithms that are both efficient and constant-time friendly. Our code is available on:
https://github.com/dj33-96/Constant-time-Quaternion-SQIsign

Acknowledgements: Anisha Mukherjee is supported by the Austrian FWF grant PAT6402023, and Sujoy Sinha Roy is partially supported by the State Government of Styria, Austria, through the Department Zukunftsfonds Steiermark. Péter Kutas is partly supported by EPSRC through grant number EP/V011324/1. Péter Kutas, Chenfeng He and Fatna Kouider are supported by the Hungarian Ministry of Innovation and Technology NRDI Office within the framework of the Quantum Information National Laboratory Program. Kutas is also supported by the grant "EXCELLENCE-151343" and by the János Bolyai Research Scholarship of the Hungarian Academy of Sciences. Andrea Basso and Sina Schaeffler are supported by SNSF Consolidator Grant CryptonIs 213766.

Bibliography

[1] TIMECOP: Automated dynamic analysis of constant-time code. https://crocs-muni.github.io/ct-tools/tutorials/timecop/, accessed: January 7, 2026

[2] Aardal, M.A., Adj, G., Aranha, D.F., Basso, A., Canales Martínez, I.A., Chávez-Saab, J., Santos, M.C., Dartois, P., De Feo, L., Duparc, M., Eriksen, J.K., Fouotsa, T.B., Filho, D.L.G., Hess, B., Kohel, D., Leroux, A., Longa, P., Maino, L., Meyer, M., Nakagawa, K., Onuki, H., Panny, L., Patranabis, S., Petit, C., Pope, G., Reijnders, K., Robert, D., Rodríguez Henríquez, F., Schaeffler, S., Wesolowski, B.: SQIsign. Tech. rep., National Institute of Standards and Technology (2024), available at https://csrc.nist.gov/Projects/pqc-dig-sig/round-2-additional-signatures

[3] Aardal, M.A., Basso, A., De Feo, L., Patranabis, S., Wesolowski, B.: A complete security proof of SQIsign. In: Kalai, Y.T., Kamara, S.F. (eds.) CRYPTO 2025, Part VI. LNCS, vol. 16005, pp. 190–222. Springer, Cham (Aug 2025). https://doi.org/10.1007/978-3-032-01887-8_7

[4] Basso, A., Dartois, P., De Feo, L., Leroux, A., Maino, L., Pope, G., Robert, D., Wesolowski, B.: SQIsign2D-West - the fast, the small, and the safer. In: Chung, K.M., Sasaki, Y. (eds.) ASIACRYPT 2024, Part III. LNCS, vol. 15486, pp. 339–370. Springer, Singapore (Dec 2024). https://doi.org/10.1007/978-981-96-0891-1_11

[5] Bernstein, D.J., Yang, B.: Fast constant-time gcd computation and modular inversion. IACR Trans. Cryptogr. Hardw. Embed. Syst. **2019**(3), 340–398 (2019). https://doi.org/10.13154/tches.v2019.i3.340-398

[6] Borin, G., Santos, M.C.R., Eriksen, J.K., Invernizzi, R., Mula, M., Schaeffler, S., Vercauteren, F.: Qlapoti: Simple and efficient translation of quaternion ideals to isogenies. Cryptology ePrint Archive, Paper 2025/1604 (2025), https://eprint.iacr.org/2025/1604

[7] Castryck, W., Decru, T.: An efficient key recovery attack on SIDH. In: Hazay, C., Stam, M. (eds.) EUROCRYPT 2023, Part V. LNCS, vol. 14008, pp. 423–447. Springer, Cham (Apr 2023). https://doi.org/10.1007/978-3-031-30589-4_15

[8] Chavez-Saab, J., Santos, M.C., De Feo, L., Eriksen, J.K., Hess, B., Kohel, D., Leroux, A., Longa, P., Meyer, M., Panny, L., Patranabis, S., Petit, C., Rodríguez Henríquez, F., Schaeffler, S., Wesolowski, B.: SQIsign. Tech. rep., National Institute of Standards and Technology (2023), available at https://csrc.nist.gov/Projects/pqc-dig-sig/round-1-additional-signatures

[9] Cohen, H.: A Course in Computational Algebraic Number Theory. Springer, New York, NY, USA (1993). https://doi.org/10.1007/978-3-662-02945-9

[10] Cornacchia, G.: Su di un metodo per la risoluzione in numeri interi dell'equazione $\sum_{h=0}^{n} C_h x^{n-h} y^h = P$. Giornale di Matematiche di Battaglini, 49:33-90 (1908)

[11] Cox, D.: References, pp. 335–342. John Wiley & Sons, Ltd (2013). https://doi.org/https://doi.org/10.1002/9781118400722.refs, https://onlinelibrary.wiley.com/doi/abs/10.1002/9781118400722.refs

[12] Dartois, P., Leroux, A., Robert, D., Wesolowski, B.: SQIsignHD: New dimensions in cryptography. In: Joye, M., Leander, G. (eds.) EUROCRYPT 2024, Part I. LNCS, vol. 14651, pp. 3–32. Springer, Cham (May 2024). https://doi.org/10.1007/978-3-031-58716-0_1

[13] Deuring, M.: Die typen der multiplikatorenringe elliptischer funktionenkörper. Abhandlungen aus dem Mathematischen Seminar der Universität Hamburg **14**, 197–272 (1941)

[14] Feo, L.D., Jao, D., Plût, J.: Towards quantum-resistant cryptosystems from supersingular elliptic curve isogenies. J. Math. Cryptol. **8**(3), 209–247 (2014). https://doi.org/10.1515/JMC-2012-0015, https://doi.org/10.1515/jmc-2012-0015

[15] Hanyecz, O., Karenin, A., Kirshanova, E., Kutas, P., Schaeffler, S.: Constant time lattice reduction in dimension 4 with application to sqisign. IACR Trans. Cryptogr. Hardw. Embed. Syst. **2025**(2), 511–534 (2025). https://doi.org/10.46586/TCHES.V2025.I2.511-534, https://doi.org/10.46586/tches.v2025.i2.511-534

[16] Jao, D., Azarderakhsh, R., Campagna, M., Costello, C., Feo, L.D., Hess, B., Hutchinson, A., Jalali, A., Karabina, K., Koziel, B., LaMacchia, B., Longa, P., Naehrig, M., Pereira, G., Renes, J., Soukharev, V., Urbanik, D.:

Supersingular isogeny key encapsulation. Tech. rep. (2022), https://sike.org/files/SIDH-spec.pdf

[17] Jao, D., Feo, L.D.: Towards quantum-resistant cryptosystems from supersingular elliptic curve isogenies. In: Yang, B. (ed.) Post-Quantum Cryptography - 4th International Workshop, PQCrypto 2011, Taipei, Taiwan, November 29 - December 2, 2011. Proceedings. Lecture Notes in Computer Science, vol. 7071, pp. 19–34. Springer (2011). https://doi.org/10.1007/978-3-642-25405-5_2, https://doi.org/10.1007/978-3-642-25405-5_2

[18] Kim, W., Lee, J., Kim, H., Lee, C.: SQIsign with fixed-precision integer arithmetic. Cryptology ePrint Archive, Paper 2025/1649 (2025), https://eprint.iacr.org/2025/1649

[19] Kirschmer, M., Voight, J.: Algorithmic enumeration of ideal classes for quaternion orders. SIAM Journal on Computing **39**(5), 1714–1747 (2010)

[20] Kocher, P., Horn, J., Fogh, A., Genkin, D., Gruss, D., Haas, W., Hamburg, M., Lipp, M., Mangard, S., Prescher, T., Schwarz, M., Yarom, Y.: Spectre Attacks: Exploiting Speculative Execution. In: 2019 IEEE Symposium on Security and Privacy (SP). pp. 1–19 (2019). https://doi.org/10.1109/SP.2019.00002

[21] Kocher, P.C., Jaffe, J., Jun, B.: Differential power analysis. In: Wiener, M.J. (ed.) CRYPTO'99. LNCS, vol. 1666, pp. 388–397. Springer, Berlin, Heidelberg (Aug 1999). https://doi.org/10.1007/3-540-48405-1_25

[22] Korzilius, S., Schoenmakers, B.: Divisions and square roots with tight error analysis from newton–raphson iteration in secure fixed-point arithmetic. Cryptography **7**(3), 43 (2023)

[23] Kouider, F., Mukherjee, A., Jacquemin, D., Kutas, P.: Constant-time integer arithmetic for SQIsign. In: Nitaj, A., Petkova-Nikova, S., Rijmen, V. (eds.) AFRICACRYPT 25. LNCS, vol. 15651, pp. 192–215. Springer, Cham (Jul 2025). https://doi.org/10.1007/978-3-031-97260-7_10

[24] Maino, L., Martindale, C., Panny, L., Pope, G., Wesolowski, B.: A direct key recovery attack on SIDH. In: Hazay, C., Stam, M. (eds.) EUROCRYPT 2023, Part V. LNCS, vol. 14008, pp. 448–471. Springer, Cham (Apr 2023). https://doi.org/10.1007/978-3-031-30589-4_16

[25] Nakagawa, K., Onuki, H., Castryck, W., Chen, M., Invernizzi, R., Lorenzon, G., Vercauteren, F.: SQIsign2D-East: A new signature scheme using 2-dimensional isogenies. In: Chung, K.M., Sasaki, Y. (eds.) ASIACRYPT 2024, Part III. LNCS, vol. 15486, pp. 272–303. Springer, Singapore (Dec 2024). https://doi.org/10.1007/978-981-96-0891-1_9

[26] Robert, D.: Breaking SIDH in polynomial time. In: Hazay, C., Stam, M. (eds.) EUROCRYPT 2023, Part V. LNCS, vol. 14008, pp. 472–503. Springer, Cham (Apr 2023). https://doi.org/10.1007/978-3-031-30589-4_17

[27] Schaeffler, S.: Algorithms for Quaternion Algebras in SQIsign. Master's thesis, ETH Zurich (2023), https://ethz.ch/content/dam/ethz/special-interest/infk/inst-infsec/appliedcrypto/education/theses/Sina_Schaeffler_Master_Thesis.pdf

[28] Scott, M.: A note on the calculation of some functions in finite fields: Tricks of the trade. Cryptology ePrint Archive, Paper 2020/1497 (2020), https://eprint.iacr.org/2020/1497
[29] Shor, P.W.: Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. SIAM J. Comput. **26**(5), 1484–1509 (1997)
[30] Shor, P.: Algorithms for quantum computation: discrete logarithms and factoring. In: Proceedings 35th Annual Symposium on Foundations of Computer Science. pp. 124–134 (1994). https://doi.org/10.1109/SFCS.1994.365700
[31] Voight, J.: Quaternion algebras. Springer Nature (2021)

A Appendix

A.1 Proof of constant-time integer square root finding algorithm

We provide a proof of correctness and constant-timeness of the integer square root finding algorithm discussed in Algorithm 3 in Section 5.1.

Theorem 3. *Let* $a \in \mathbb{N}$, $a = qx_n + r$, *where* $q = \left\lfloor \frac{a}{x_n} \right\rfloor$ *and* $0 \leq r < x_n$,, *define the sequence* (x_n) *by,*

$$x_{n+1} = \left\lfloor \frac{x_n + \left\lfloor \frac{a}{x_n} \right\rfloor}{2} \right\rfloor = \left\lfloor \frac{x_n + q}{2} \right\rfloor, \textit{with} \quad x_0 \geq \sqrt{a}.$$

Then (x_n) *converges to the integer square root of* a*, i.e.,*

$$\lim_{n \to \infty} x_n = \lfloor \sqrt{a} \rfloor.$$

Furthermore, whenever $n > log_2(x_0)$*, the sequence* $\{x_n\}$ *becomes constant, thus in particular Algorithm 3 is correct and runs in constant time.*

Proof. Let denote $s = \lfloor \sqrt{a} \rfloor$. Then,

$$s^2 \leq a < (s+1)^2.$$

Claim 1: $x_{n+1} - x_n \geq 0$.

Proof. Since $x_n \geq \sqrt{a}$, we have:

$$q_n = \left\lfloor \frac{a}{x_n} \right\rfloor \leq x_n \Rightarrow x_n + q_n \leq 2x_n \Rightarrow \left\lfloor \frac{x_n + q_n}{2} \right\rfloor \leq x_n \Rightarrow x_{n+1} \leq x_n$$

Thus, $x_{n+1} - x_n \leq 0$.

□

Claim 2: $x_n - \lfloor\sqrt{a}\rfloor \geq 0 : \forall n \in \mathbb{N},\, x_n \geq \lfloor a \rfloor$.

Proof. We prove this claim by induction.

- Our initial term x_0 satisfies $x_0 \geq \lfloor\sqrt{a}\rfloor$.
- Now, Since $x_n \geq \lfloor\sqrt{a}\rfloor$, we have $\left\lfloor \frac{a}{x_n} \right\rfloor \leq \left\lfloor \frac{a}{\lfloor\sqrt{a}\rfloor} \right\rfloor$; and thus both x_n and $\left\lfloor \frac{a}{x_n} \right\rfloor$ are greater than or equal to $\lfloor\sqrt{a}\rfloor$.
- Now consider x_{n+1}:

$$x_{n+1} = \left\lfloor \frac{x_n + \left\lfloor \frac{a}{x_n} \right\rfloor}{2} \right\rfloor.$$

  Let $b = \lfloor\sqrt{a}\rfloor$ and write $x_n = b + d$ and $q_n = b - e$. Now we have to show that $d \geq e$. One can assume that $e > 0$ as otherwise the statement immediately follows. We have that

$$a = x_n q_n + r_n$$

  and that $x_n \geq b$ and $b^2 < a$. Furthermore, by the properties of division with remainder $r_n < b + d$. This means that $b^2 < (b + d)(b - e) + r_n$. This can be rearranged as

$$(e - d)b + de < r_n.$$

  Now if $e > d$, then by the fact that $e > 0$, we have a contradiction as the left hand side is at least $b + d$ which should be smaller than r_n.

By induction, we have shown that $x_n \geq \lfloor\sqrt{a}\rfloor$ for all n. Therefore, the sequence (x_n) bounded below by $\lfloor\sqrt{a}\rfloor$, and we have: $x_n \geq \lfloor\sqrt{a}\rfloor$ for all n. This in particular shows now that the sequence $\{x_n\}$ converges. □

Claim 3: The sequence $\{x_n\}$ converge to $\lfloor\sqrt{a}\rfloor$ and stabilize.

Proof. Now the above notation allows one to see that $x_{n+1} = b + \left\lfloor \frac{d-e}{2} \right\rfloor$. This shows that at every step the distance of x_n to $b = \lfloor\sqrt{a}\rfloor$ halves with shows that x_n has to converge to $\lfloor\sqrt{a}\rfloor$. Furthermore, after $log_2(k)$ iterations (k is the bitsize of a), the sequence will eventually reach $\lfloor\sqrt{a}\rfloor$ and stabilize. □

A.2 ct-RepresentInteger

This section contains the algorithm for the ct-RepresentInteger$_{\mathcal{O}}$. The expressions for z_k and t_k can be found in Section 5.3.

Algorithm 9 $\mathsf{ct{-}RepresentInteger}_{\mathcal{O}}(M, \omega)$

Input: $M \in \mathbb{Z}$ such that $M > 2^{32} \cdot p$ and $M = u(2^v - u)$, with u a random odd integer.
Input: $\omega \in B_{p,\infty}$ such that $q = -\omega^2$ is a positive integer and $q = 1 \mod 4$.
Input: $\mathcal{O}$ a special extremal maximal order containing the suborder $\mathbb{Z}[\omega] + j \cdot \mathbb{Z}[\omega] \subset \mathcal{O}$, with j from the standard basis of $B_{p,\infty}$.
Output: $\gamma \in \mathcal{O}$ with $n(\gamma) = M$
Output: found, a boolean indicating whether a solution was found

Set $(x_f, y_f, z_f, t_f) = (0, 0, 0, 0)$
Set $q = -\omega^2$ and found $= 0$
Set $d = M \mod 3$, $q_3 = q \mod 3$ and $q_8 = q \mod 8$
for $k = 1$ to l **do** ▷ l is defined in Section 5.4
    Set $z = z_k(k, d, q_3, q_8)$ ▷ See Equation (3)
    Set $t = t_k(k, d)$ ▷ See Equation (4)
    Set $N = 4M - p \cdot (z^2 + qt^2)$
    Compute $u_0 = \mathsf{ct\text{-}Tonelli\text{-}Shanks}(N - q, N)$ ▷ Algorithm 6
    Compute $x = \mathsf{ctCornacchia}(N, u_0, q, \sqrt{N})$
    Compute $y = \sqrt{\frac{N - x^2}{q}}$
    Compute $\gamma = \mathsf{make\text{-}quaternion\text{-}primitive}(x, y, z, t, \mathcal{O})$
    Compute diff $= M - (\gamma[0]^2 + q \cdot \gamma[1]^2 + p(\gamma[2]^2 + q \cdot \gamma[3]^2)$
    Set $s = \mathsf{ctIsZero}(\text{diff})$ ▷ See Section 2
    Set $(x_f, y_f, z_f, t_f) = (\gamma[0], \gamma[1], \gamma[2], \gamma[3])$ ▷ Using $s = 1$ in a ctCondMov
    found $= \mathsf{ctCondMov}(1, s, s)$
return found, $\gamma = x_f + y_f\omega + z_f j + t_f \omega j \in \mathcal{O}$